

Documento de Planificación del TFG: Sistema de Visualización y Análisis de Biomarcadores Multi-Usuario

Alumna: Inés

Tutor: Alberto

Proyecto: Plataforma para la gestión, procesamiento y visualización de datos provenientes de dispositivos wearables (enfocado en EmbracePlus).

1. Arquitectura del Sistema (Stack Tecnológico)

Se ha seleccionado un stack orientado a la eficiencia en el manejo de series temporales de alta frecuencia y la reactividad multiplataforma.

A. Persistencia de Datos: PostgreSQL + TimescaleDB

- **Por qué:** Los datos de la EmbracePlus (acelerómetro, EDA, pulso) son series temporales puras que dependen de un *timestamp*.
- **Justificación técnica:**
 - **Uso de Hypertables:** TimescaleDB particiona los datos por tiempo automáticamente. Esto es vital dado que sensores como el acelerómetro pueden muestrear hasta a 208 Hz, generando millones de registros rápidamente.
 - **Integridad Relacional:** Se mantiene la potencia de SQL para cruzar el `participant_full_id` con metadatos de usuario y configuraciones personalizadas.

B. Backend y Procesamiento: Python + FastAPI

- **Justificación técnica:**
 - **Asincronía (FastAPI):** Ideal para recibir flujos constantes de datos de varios usuarios sin bloquear la ejecución del servidor.
 - **Pipeline ETL Avanzado:** Utilizando Pandas/NumPy, el backend filtrará automáticamente etiquetas de desuso como `device_not_recording`.
 - **Resampleado de Datos:** El sistema implementará algoritmos para alinear métricas con frecuencias dispares (Temp a 1-4 Hz vs PPG a 208 Hz) en un mismo eje temporal.
 - **Patrón Adapter:** Arquitectura modular diseñada para permitir la integración futura de otros dispositivos (Fitbit, Apple Watch) sin cambiar la lógica de negocio.

C. Frontend y Visualización: Flutter (Dart)

- **Justificación técnica:**

- **Multiplataforma:** Capacidad de desplegar como Web Dashboard y App Móvil con un único código base.
- **Renderizado Reactivo:** Actualización en tiempo real de gráficas complejas sin caídas de *frames* (60fps), garantizando fluidez en datos de alta densidad.
- **Gestión de Estado (Riverpod/Bloc):** Control eficiente de múltiples flujos de datos independientes para visualizar a varios usuarios a la vez de forma reactiva.

2. Aportes de Ingeniería (Valor Añadido)

Para diferenciar este proyecto de una simple visualización, se implementarán:

1. **Módulo de Fiabilidad del Dato:** Uso de metadatos del dispositivo para marcar visualmente registros con `low_signal_quality` o `worn_during_motion`, avisando si el dato es dudoso.
2. **Correlación Inteligente:** Análisis combinado de EDA (Actividad Electrodermica) y HR (Frecuencia Cardíaca) para distinguir entre estrés emocional y esfuerzo físico.
3. **Gestión de Sincronización Diferida:** Control de la ingesta masiva de paquetes históricos cuando el dispositivo vuelca la memoria interna (128 MB) tras horas de uso offline.
4. **Sistema de Alertas:** Notificaciones en el dashboard basadas en umbrales de salud y estados del dispositivo (batería baja o memoria llena).

3. Hoja de Ruta (Roadmap de Trabajo)

Fase 1: Análisis y ETL (Semanas 1-2)

- Análisis de la estructura de los CSV (columnas `timestamp_iso`, `participant_id`, métricas).
- Desarrollo de scripts de limpieza y filtrado de estados de carga/desuso.
- Investigación de técnicas de resampling para normalizar frecuencias.

Fase 2: Infraestructura y Backend (Semanas 3-5)

- Despliegue de PostgreSQL con la extensión TimescaleDB.
- Creación de la API para ingesta y consulta de biomarcadores (Documentación automática con Swagger).
- Diseño del modelo de datos jerárquico (Participante -> Sesión -> Métricas).

Fase 3: Dashboard en Flutter (Semanas 6-9)

- Diseño de la interfaz multi-usuario reactiva.
- Integración de gráficas avanzadas para series temporales (fl_chart).
- Implementación de filtros dinámicos por rango de fechas y participantes.

Fase 4: Refinamiento e Innovación (Semanas 10+)

- Módulo de detección de calidad de señal basado en el manual técnico.
- Generación de alertas automáticas.
- Redacción de la memoria técnica final.

4. Próximos Pasos Inmediatos

1. **Script de Ingesta Prototipo:** Python script para leer un CSV y cargar los datos útiles (post-fila 752) en una tabla local.
2. **Entorno Docker:** Configurar un contenedor con la base de datos y la API para asegurar la portabilidad.
3. **Mockup Flutter:** Diseñar la "Tarjeta de Usuario" que servirá de unidad básica para el dashboard multi-usuario.